

Learning Objectives

1. String manipulation.
2. Use a list as a data type to store multiple elements.
3. Implement simple functions in Python.

Estimated Size

3 program file.

Credit

Failing to submit this assignment will **not** result in a decrease in your lab grade. These assignments are **optional extra credit assignments**.

Setup

In your Python files, you must include author information. On the first line, add a **# Author comment line**, where you mention your full name. Similarly, include **your email and Spire ID** on the **# Email** and **# Spire ID** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678
```

1. Implement **ab.py** (5 points)

This is a new program you must create from scratch. In VSCode, create a new file and name it **ab.py**. Note that the file name is **case-sensitive**: all letters must be lower-case. Naming it **Ab.py** or **AB.py** would NOT work. Next, include the three lines of comments with author info at the beginning of this file.

The task of this program is to read in two inputs from the user (using the input function):

1. An integer (let's call it **a**)
2. An integer (let's call it **b**)

The program should then output the following pattern:

- The number **a** repeated **b** times, followed by the number **b** repeated **a** times

Running a correct implementation of this program should look like:

```
Enter the number a: 5
Enter the number b: 3
55533333
```

Another example:

```
Enter the number a: 4
Enter the number b: 7
44444447777
```

The prompt text/flavor text of your `input()` function does not have to exactly match the example above. The autograder does not check it, so you can use any reasonable prompt text as you want. **But make sure the output is only as asked. Do not output anything extra.** After finishing the program, run it a few times, try different inputs, and verify the outputs are correct.

2. Implement `list.py` (5 points)

Create a new program, name it `list.py`. Next, include the three lines of comments at the beginning of this file. Next, read the inputs as follows:

- A string (`a`)
- A string (`b`)
- A string (`c`)
- A string (`d`)

The program should then do the following:

- Create an empty list `lst`.
- Add `a` at the end of `lst`.
- Print `lst`.
- Add `b` at the end of `lst`.
- Print `lst`.
- Add `c` at the beginning of `lst`.
- Print `lst`.
- Remove `d` from `lst`. You should assume that `d` is already in `lst`. (Think about the behavior of `remove`, why does this work?)
- Print `lst`.
- Print the length of `lst` using `len()` function.

Example:

```
Hello
Hola
Namaste
Hola
['Hello']
```

```
['Hello', 'Hola']
['Namaste', 'Hello', 'Hola']
['Namaste', 'Hello']
2
```

3. Implement `ab_with_func.py` (5 points)

In this problem, we will try out the custom functions made by you. You need to implement a function named `ab_func()` with two parameters, `a` and `b`, which are two integers. Then your function needs to **RETURN** a string with the following pattern

- The number `a` repeated `b` times, followed by the number `b` repeated `a` times.

Assuming you have implemented `ab_func()` correctly, below are some examples of calling the function and the expected return values.

```
print(ab_func(5, 3))
55533333

print(ab_func(4, 7))
44444447777
```

You may use the following starter code:

```
# AUTHOR    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678

# Below, please implement the ab_func() function as specified in the pre-lab
# instructions.
# ----- YOUR CODE STARTS HERE -----

# ===== YOUR CODE ENDS HERE =====

# You can use the code below to help test your functions

# Uncomment the following lines (i.e. remove the # characters on each line)
# to test ab_func()
#print(ab_func(5, 3))
#print(ab_func(4, 7))
```

4. Submit your code to Gradescope

Log in to [Gradescope](#), select **Extra Credit Practice Lab 02**, and submit **ab.py**, **list.py**, and **ab_with_func.py**. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions, which will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the [UMass Academic Honesty Policy and Procedures](#) so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty, you should re-read the course policies. We assume you have read the course policies in detail, and by submitting this project, you have provided your virtual signature in agreement with these policies.